

DEVELOPER ONBOARDING · PART 2

ReactJS Authentication, Performance & Coding Standards

A continuation of the React Developer Onboarding hands-on project

Prepared for: New Intern / Developer Onboarding

Continues from: ReactJS Developer Prerequisite & Hands-on Learning Guide (Doc 1)

Project used: The react-learning project you already created

Goal: Add real authentication, pagination, and performance handling — the right way

Contents

- 1. Before You Start
- 2. Authentication – Login & Token
- 3. Pagination
- 4. Frontend Performance – Handling Large Lists
- 5. Coding Standards & Best Practices
- 6. Troubleshooting Guide
- 7. Suggested Additional Topics

1. Before You Start

This guide continues directly from **Doc 1 — ReactJS Developer Prerequisite & Hands-on Learning Guide**. Keep using the same react-learning project you set up there — don't start a new one.

What you should already have

- Completed Hands-on 1-9 from Doc 1 (JSX, components, props, state, forms, lists, `useEffect`, calling an API, React Router)
- `react-learning` running locally with `npm run dev`
- `react-router-dom` already installed

Additional packages used in this guide

```
npm install axios react-window
```

Package	Used for
axios	API calls with request/response interceptors (Section 2, 3)
react-window	Rendering large lists efficiently (Section 4)

The sample API used in this guide

Instead of a fake, unreachable API, this guide uses a real hosted test API: **ReqRes** (`reqres.in`). It gives you real HTTP responses, a real login endpoint, and a real paginated users endpoint — good practice for working with an actual backend.

ReqRes requires a free API key on every request. Sign up at `https://reqres.in/signup` and copy your key before continuing — without it every call returns 401 `missing_api_key`.

Set up your environment variables

Create a `.env` file in the root of `react-learning`:

```
VITE_API_BASE_URL=https://reqres.in/api
VITE_API_KEY=paste_your_reqres_api_key_here
```

Add `.env` to `.gitignore` if it isn't already there — never commit real keys.

2. Authentication — Login & Token

Most real applications authenticate users with a token-based flow. The typical sequence:

1. You submit the login form (email/password)
2. The backend validates the credentials and returns an **access token**
3. The frontend stores the token and attaches it to every future API request as an `Authorization: Bearer <token>` header
4. Protected routes check whether you are authenticated before rendering
5. When the token is rejected (expired/invalid), you are sent back to the login page

Sample login endpoint

Method	POST
URL	https://reqres.in/api/login
Headers	x-api-key: <your key>, Content-Type: application/json
Test credentials	email: eve.holt@reqres.in, password: cityslicka

Try it with curl first, so you know what a correct response looks like before you write any React code:

```
curl -X POST https://reqres.in/api/login \
-H "Content-Type: application/json" \
-H "x-api-key: YOUR_API_KEY" \
-d '{"email":"eve.holt@reqres.in","password":"cityslicka"}'
```

Successful response:

```
{
  "token": "QpwL5tke4Pnpja7X4"
}
```

If you send a wrong/missing field, ReqRes returns a `400` with an error message — useful later for testing your error-handling UI.

Where to store the token

Storage	Pros	Cons
httpOnly cookie	Not readable by JavaScript, safer against XSS	Needs CSRF protection; requires backend cookie support
localStorage	Simple, works with any backend	Readable by any script — vulnerable if the app has an XSS bug

Prefer an httpOnly cookie for production apps when the backend supports it. This guide uses localStorage because ReqRes is a client-only demo API — understand the tradeoff before using it in a real product.

1 API Client with Interceptors

Create `src/services/apiClient.ts`. Every request automatically gets the ReqRes API key and, once you're logged in, your access token.

```
// src/services/apiClient.ts
import axios from "axios";

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: {
    "x-api-key": import.meta.env.VITE_API_KEY,
  },
});

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem("accessToken");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("accessToken");
    }
  }
);
```

```

        window.location.href = "/login";
    }
    return Promise.reject(error);
  }
});

export default apiClient;

```

2 Auth Service

```

// src/services/authService.ts
import apiClient from "../apiClient";

interface LoginPayload {
  email: string;
  password: string;
}

interface LoginResponse {
  token: string;
}

export async function login(payload: LoginPayload): Promise<LoginResponse> {
  const { data } = await apiClient.post<LoginResponse>("/login", payload);
  return data;
}

export function logout() {
  localStorage.removeItem("accessToken");
}

```

3 Auth Context

```

// src/context/AuthContext.tsx
import { createContext, useContext, useState, ReactNode } from "react";
import { login as loginRequest, logout as logoutRequest } from "../services/authService";

interface AuthContextType {
  email: string | null;
  isAuthenticated: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [email, setEmail] = useState<string | null>(
    localStorage.getItem("userEmail")
  );

  const login = async (emailInput: string, password: string) => {
    const data = await loginRequest({ email: emailInput, password });
    localStorage.setItem("accessToken", data.token);
    localStorage.setItem("userEmail", emailInput);
    setEmail(emailInput);
  };

  const logout = () => {
    logoutRequest();
    localStorage.removeItem("userEmail");
    setEmail(null);
  };

  return (
    <AuthContext.Provider value={{ email, isAuthenticated: !!email, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
}

```

```
    return context;
  }
}
```

Wrap your app once in `main.tsx` :

```
<AuthProvider>
  <App />
</AuthProvider>
```

Your Task — Build the Login Page

Now build `src/pages/Login.tsx` yourself. Don't look at the reference solution until you've tried.

Requirements:

- An email input and a password input, both controlled with `useState`
- A submit handler that calls `useAuth().login(email, password)`
- While the request is in flight, disable the submit button and show a loading state
- If login fails, show an error message (test this using a wrong password)
- On success, navigate to `/dashboard` using `useNavigate()`
- Test with the ReqRes credentials above, and confirm the token appears in `localStorage` (DevTools → Application → Local Storage)

Reference Solution

```
// src/pages/Login.tsx
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (event: React.FormEvent) => {
    event.preventDefault();
    setError("");
    setLoading(true);

    try {
      await login(email, password);
      navigate("/dashboard");
    } catch (err) {
      setError("Invalid email or password");
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      {error && <p>{error}</p>}
      <button type="submit" disabled={loading}>
        {loading ? "Logging in..." : "Login"}
      </button>
    </form>
  );
}

export default Login;
```

```
// src/components/PrivateRoute.tsx
import { Navigate, Outlet } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

function PrivateRoute() {
  const { isAuthenticated } = useAuth();
  return isAuthenticated ? <Outlet /> : <Navigate to="/login" replace />;
}

export default PrivateRoute;
```

Usage in the router:

```
<Routes>
  <Route path="/login" element={<Login />} />

  <Route element={<PrivateRoute />>
    <Route path="/dashboard" element={<Dashboard />} />
  </Route>
</Routes>
```

Token security best practices

- Keep access tokens short-lived and use a refresh token for longer sessions when the backend supports one
- Never store sensitive data inside a JWT payload — it is base64-encoded, not encrypted, and anyone can decode it
- Always use HTTPS in production so tokens are never exposed in transit
- Clear all stored tokens on logout and on any 401 response
- Validate token expiry on both the frontend (for UX) and the backend (for security) — never trust a frontend-only check

3. Pagination

There are two different ways to paginate a list, and picking the wrong one is a common mistake. Both are covered below with a working sample.

3.1 Client-side pagination

What it is: the frontend fetches the *entire* dataset once, then slices it into pages in the browser using JavaScript. No extra network request is made when you change pages.

Use it when	The dataset is small (roughly under 500-1000 rows), doesn't change often, and is cheap to fetch all at once
Avoid it when	The dataset is large, grows over time, or needs server-side filtering/sorting/search

A Client-side Pagination Sample

```
// src/hooks/useClientPagination.ts
import { useMemo, useState } from "react";

export function useClientPagination<T>(data: T[], pageSize = 10) {
  const [page, setPage] = useState(1);
  const totalPages = Math.max(1, Math.ceil(data.length / pageSize));

  const pageData = useMemo(() => {
    const start = (page - 1) * pageSize;
    return data.slice(start, start + pageSize);
  }, [data, page, pageSize]);

  return { page, setPage, totalPages, pageData };
}
```

Usage:

```
const { page, setPage, totalPages, pageData } = useClientPagination(allUsers, 10);

return (
  <>
    {pageData.map((u) => <div key={u.id}>{u.name}</div>)}

    <button disabled={page === 1} onClick={() => setPage(page - 1)}>Prev</button>
    <span>Page {page} / {totalPages}</span>
    <button disabled={page === totalPages} onClick={() => setPage(page + 1)}>Next</button>
  </>
);
```

3.2 Server-side pagination

What it is: the frontend asks the backend for one page's worth of data at a time. The backend does the slicing (and usually filtering/sorting too) and returns a small response plus metadata about how many pages exist.

Use it when	The dataset is large or grows over time, and you need consistent performance regardless of total size
Avoid it when	The dataset is tiny — the extra network round-trip per page isn't worth it

Sample server-pagination endpoint

Method	GET
URL	https://reqres.in/api/users?page=2
Headers	x-api-key: <your key>

Response — note it returns pagination metadata alongside the page of data:

```
{
  "page": 2,
  "per_page": 6,
  "total": 12,
  "total_pages": 2,
  "data": [
    { "id": 7, "email": "michael.lawson@reqres.in", "first_name": "Michael", "last_name": "Lawson" },
    ...
  ]
}
```

Your Task — Fetch a Page from the API

Write a hook `useUsersPage(page: number)` that calls `GET /users?page={page}` using `apiClient` from Section 2, and returns `{ data, total, totalPages, loading, error }`. Re-fetch whenever `page` changes.

Reference Solution

```
// src/hooks/useUsersPage.ts
import { useEffect, useState } from "react";
import apiClient from "../services/apiClient";

interface User {
  id: number;
  email: string;
  first_name: string;
  last_name: string;
}

interface UsersPageResponse {
  page: number;
  total: number;
  total_pages: number;
  data: User[];
}

export function useUsersPage(page: number) {
  const [data, setData] = useState<User[]>([]);
  const [total, setTotal] = useState(0);
  const [totalPages, setTotalPages] = useState(1);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    setError("");

    apiClient
      .get<UsersPageResponse>(`/users?page=${page}`)
      .then((res) => {
        if (cancelled) return;
        setData(res.data.data);
        setTotal(res.data.total);
        setTotalPages(res.data.total_pages);
      })
      .catch(() => {
        if (!cancelled) setError("Failed to load users");
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });

    return () => {
      cancelled = true;
    };
  }, [page]);

  return { data, total, totalPages, loading, error };
}
```

Usage:

```
const [page, setPage] = useState(1);
const { data, totalPages, loading, error } = useUsersPage(page);

if (loading) return <p>Loading...</p>;
if (error) return <p>{error}</p>;

return (
  <>
    {data.map((u) => <div key={u.id}>{u.first_name} {u.last_name}</div>)}

    <button disabled={page === 1} onClick={() => setPage((p) => p - 1)}>Prev</button>
    <span>Page {page} / {totalPages}</span>
    <button disabled={page === totalPages} onClick={() => setPage((p) => p + 1)}>Next</button>
  </>
);
```

Client-side vs. server-side — quick comparison

	Client-side	Server-side
Network requests	One, upfront	One per page
Works well with	Small, static datasets	Large or growing datasets
Filtering/sorting cost	Cheap — done in-browser	Needs backend support
Initial load time	Slower if dataset is large	Fast — only loads one page
Always in sync with backend	No — stale until refetched	Yes — fresh on every page

4. Frontend Performance — Handling Large Lists

Rendering a few dozen list items is never a problem. Rendering thousands of rows — an activity feed, a big table, a long dropdown — can freeze the UI if you don't plan for it. This section covers the standard techniques, in the order you should reach for them.

Why large lists get slow

- Every rendered row is a real DOM node — thousands of DOM nodes are expensive for the browser to create, layout, and repaint
- A single state change can re-render the whole list if items aren't memoized
- Inline functions/objects passed as props create a new reference on every render, defeating memoization

Technique 1 — Memoize list items with `React.memo`

```
const UserRow = React.memo(function UserRow({ user }: { user: User }) {  
  return <div>{user.first_name} {user.last_name}</div>;  
});
```

`React.memo` skips re-rendering `UserRow` if its props haven't changed — but only if the props you pass are referentially stable (see the next two techniques).

Technique 2 — `useMemo` and `useCallback`

```
// Recompute the filtered list only when users or query change  
const filteredUsers = useMemo(  
  () => users.filter((u) => u.first_name.includes(query)),  
  [users, query]  
);  
  
// Keep the same function reference across renders  
const handleSelect = useCallback((id: number) => {  
  setSelectedId(id);  
}, []);
```

Without `useCallback`, an inline `onClick={() => handleSelect(user.id)}` creates a brand-new function on every render, which breaks `React.memo` on the row that receives it.

Technique 3 — Stable, unique key props

- Always use a stable unique id as the `key` — never the array index if the list can be filtered, sorted, or reordered
- A wrong or unstable key causes React to unnecessarily unmount/remount rows, which is expensive and can cause bugs (e.g. losing input focus)

Technique 4 — List virtualization (windowing)

For genuinely large lists (1,000+ rows), don't render rows that aren't visible on screen. `react-window` renders only the visible slice and recycles DOM nodes as you scroll.

```
npm install react-window
```

```
// src/components/VirtualUserList.tsx  
import { FixedSizeList as List } from "react-window";  
  
interface User {  
  id: number;  
  first_name: string;  
  last_name: string;  
}  
  
function VirtualUserList({ users }: { users: User[] }) {  
  return (  
    <List  
      height={480}  
      width="100%"  
      itemCount={users.length}  
      itemSize={40}  
    >  
      {({ index, style }) => (  
        <div style={style}>  
          {users[index].first_name} {users[index].last_name}  
        </div>  
      )}  
    </List>  
  );  
}
```

```
export default VirtualUserList;
```

Even with 50,000 rows in `users`, only the ~12 rows visible in the 480px viewport are ever mounted in the DOM.

Technique 5 — Debounce search/filter input

If typing in a search box filters a large list (or calls an API) on every keystroke, debounce it so filtering only runs after the user pauses typing.

```
// src/hooks/useDebouncedValue.ts
import { useEffect, useState } from "react";

export function useDebouncedValue<T>(value: T, delayMs = 300): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(timer);
  }, [value, delayMs]);

  return debounced;
}
```

```
const [query, setQuery] = useState("");
const debouncedQuery = useDebouncedValue(query, 300);

// use debouncedQuery to filter or call the API – not query directly
```

Technique 6 — Code splitting / lazy loading

Don't ship code the user hasn't asked for yet. Split large or rarely-visited pages out of the initial bundle:

```
import { lazy, Suspense } from "react";

const Reports = lazy(() => import("../pages/Reports"));

<Suspense fallback=<p>Loading...</p>>
  <Reports />
</Suspense>
```

Which technique to reach for

Symptom	Technique
List re-renders on unrelated state changes	<code>React.memo</code> + <code>useCallback/useMemo</code>
Rows lose state / re-mount unexpectedly	Fix the key prop
1,000+ rows, scrolling feels laggy	Virtualization (<code>react-window</code>)
Typing in search feels laggy / spams the API	Debounce the input
Initial page load is slow	Code splitting / lazy loading

5. Coding Standards & Best Practices

Naming conventions

Type	Convention	Example
Component file	PascalCase.tsx	UserCard.tsx, LoginForm.tsx
Custom hook file	camelCase, prefixed use	useAuth.ts, useUsersPage.ts
Service file	camelCase + Service	authService.ts
Type/interface file	camelCase, singular	user.ts
Variables / functions	camelCase	getUserData(), isLoading
Constants	UPPER_SNAKE_CASE	API_BASE_URL
Booleans	prefixed is/has/should	isLoggedInIn, hasError

Project folder structure

Apply this structure to `react-learning` going forward — it's the same shape you'll use on the real project:

```
src/
├── components/    # Reusable UI components (UserRow, PrivateRoute...)
├── pages/         # Route-level components (Login, Dashboard...)
├── hooks/         # Custom hooks (useAuth, useUsersPage, useDebounceValue...)
├── services/      # API calls (apiClient, authService...)
├── context/       # React Context providers (AuthContext...)
├── types/         # TypeScript types / interfaces
├── utils/         # Helper functions
└── constants/    # App-wide constants
```

Component best practices

- One component per file
- Keep components small and focused on a single responsibility
- Extract logic into a custom hook once it grows complex (as with `useUsersPage` in Section 3)
- Destructure props in the function signature
- Wrap list-row components in `React.memo` once the list can grow large (Section 4)

TypeScript best practices

- Avoid `any` — use `unknown` and narrow it, or define a proper type
- Define an interface for every component's props and every API response shape
- Use `type` for unions, `interface` for object shapes
- Enable `strict` mode in `tsconfig.json`

State management guidelines

- Keep state as local as possible
- Lift state up only when more than one component actually needs it
- Avoid prop drilling beyond 2–3 levels — reach for Context instead (as with `AuthContext`)
- Use Context for global, rarely-changing data (auth user, theme)
- Use a dedicated library (Zustand / Redux Toolkit) for complex, frequently-updated state

Code style

- Use Prettier and ESLint — never hand-format code
- No unused variables or imports
- Prefer `const` over `let`; never use `var`
- Use optional chaining (`?.`) and nullish coalescing (`??`)
- Keep functions short — extract a function once it does more than one thing

Environment variables

- Never hardcode API URLs or keys in code — this includes the ReqRes key you used in Section 2
- Store them in `.env` — see the setup in Section 1

- Never commit a `.env` file containing real secrets — add it to `.gitignore`
- Access it via `import.meta.env.VITE_*`

Error handling

- Always handle loading, error, and empty states for every API call (as in `useUsersPage`)
- Wrap async calls in try/catch
- Show a user-friendly error message; log the technical detail for debugging
- Add an Error Boundary to catch unexpected render errors

Git commit convention

Use [Conventional Commits](#):

```
feat: add login page
fix: resolve token refresh bug
perf: virtualize the user list
chore: update dependencies
refactor: extract useAuth hook
docs: update README
```

Code review checklist

- ☐ Code builds and lints without errors
- ☐ No `console.log` left in the code
- ☐ No hardcoded secrets or URLs
- ☐ Components are reasonably small and focused
- ☐ Proper TypeScript types — no unnecessary any
- ☐ Loading / error states are handled
- ☐ No unused imports or variables
- ☐ Large lists use stable keys and are memoized where needed

6. Troubleshooting Guide

When something breaks, the fix is almost always visible in the browser DevTools — you just need to know where to look. This section is your first stop before asking for help.

Open DevTools

- Chrome/Edge: right-click anywhere on the page → **Inspect**, or press `F12` / `Ctrl+Shift+I` (`Cmd+Option+I` on Mac)
- Install the **React Developer Tools** browser extension — it adds "Components" and "Profiler" tabs for inspecting props/state directly

6.1 Reading frontend (console) errors

Open the **Console** tab. Read the first line of red text — that's the actual error. Click the file/line reference on the right to jump straight to the source.

Error message	What it means	How to fix
Cannot read properties of undefined (reading 'x')	You accessed a property on something that is undefined/null	Check the data exists first, or use optional chaining: <code>user?.name</code>
Objects are not valid as a React child	You tried to render an object/array directly in JSX	Render a specific field, e.g. <code>{user.name}</code> , not <code>{user}</code>
Each child in a list should have a unique "key" prop	A <code>.map()</code> is missing the key prop	Add a stable unique id as key
Maximum update depth exceeded	A state update is triggering itself in a loop	Check your <code>useEffect</code> dependency array; don't call <code>setState</code> unconditionally in the component body
Invalid hook call / Hooks can only be called inside a function component	A hook was called conditionally, in a loop, or outside a component	Move the hook call to the top level of the component
Uncaught (in promise) ...	An async call's rejection was never handled	Add a <code>try/catch</code> or <code>.catch()</code> around the async call

6.2 Reading API errors — the Network tab

Open the **Network** tab, filter by **Fetch/XHR**, then trigger the action that calls the API (e.g. submit the login form). Click the request that appears.

1. **Status** — the response status code (see table below)
2. **Headers** tab — confirm your request actually sent what you expect: is `Authorization` present? Is `x-api-key` present?
3. **Payload/Request** tab — the exact JSON body you sent; compare it against what the API expects
4. **Response/Preview** tab — the exact JSON the backend sent back, including its error message
5. **Timing** tab — useful if the request feels slow rather than broken

HTTP status code quick reference

Code	Meaning	Typical cause
200 / 201	Success	—
400	Bad Request	Missing or invalid field in the request body
401	Unauthorized	Missing/expired/invalid token or API key
403	Forbidden	Authenticated, but not allowed to do this
404	Not Found	Wrong URL/endpoint, or the resource ID doesn't exist
422	Unprocessable Entity	Validation failed on otherwise well-formed data
429	Too Many Requests	Rate limited — you're calling the API too fast
500	Internal Server Error	A bug on the backend — not something you can fix from the frontend
502 / 503	Bad Gateway / Service Unavailable	The backend or a service it depends on is down

6.3 CORS errors

A CORS error in the console looks like: *"has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header..."*. It means the browser blocked the response because the backend didn't explicitly allow your frontend's origin.

- This is a **backend configuration issue** — you cannot fix it by changing frontend code alone
- Confirm the request URL is correct (a typo'd domain looks like a CORS error)
- In local development, ask the backend team to allow `http://localhost:5173`, or use a dev proxy in `vite.config.ts` as a temporary

workaround

6.4 General debugging workflow

- ☐ Reproduce the issue — know the exact steps that trigger it
- ☐ Check the Console tab for a frontend error and read the stack trace
- ☐ Check the Network tab for the related API call
- ☐ Compare the actual request (headers/payload) against what the API expects
- ☐ Compare the actual response against what your code expects
- ☐ Narrow it down with `console.log` or breakpoints (click the line number in the Sources tab)
- ☐ Fix it, then verify both the success case and the error case still work

7. Suggested Additional Topics

Once you're comfortable with everything above, these are natural next steps — not required to finish this guide, but worth learning as you take on more of the real project.

Data & state

- **TanStack Query (React Query)** — replaces hand-written `useEffect` fetching with caching, automatic retries, and background refetching
- **Zustand / Redux Toolkit** — for state that's too complex or too frequently updated for Context alone
- **React Hook Form + Zod** — schema-based form validation, far less boilerplate than manual `useState` per field

Quality & reliability

- **Testing** — Vitest + React Testing Library for unit and component tests
- **Error tracking** — Sentry or similar, so production errors are visible instead of silent
- **CI/CD basics** — running lint, build, and tests automatically on every pull request

User experience

- **Accessibility (a11y)** — semantic HTML, ARIA attributes, keyboard navigation, color contrast
- **Internationalization (i18n)** — structuring text for multiple languages from the start
- **A component library / design system** — e.g. shadcn/ui or MUI, for consistent UI without reinventing every component

Security & deployment

- **Frontend security beyond tokens** — sanitizing user input, avoiding `dangerouslySetInnerHTML`, Content-Security-Policy basics
- **Production build & deployment** — `npm run build`, hosting on Vercel/Netlify, or containerizing with Docker
- **Performance profiling** — the React DevTools Profiler tab, to measure (not just guess) what's actually slow

Continue applying these standards in the same `react-learning` project — consistency here is what makes the transition to the real project smooth.